



ABTRACCIÓN

30 SEP 2025

<https://github.com/juanvarco95>

ABSTRACCIÓN

Enfocarse en lo esencial de un objeto y ocultar los detalles innecesarios

- Nos permite definir qué hace un objeto, sin preocuparnos todavía de cómo lo hace
- Sirve para modelar conceptos del mundo real en clases y métodos.

```
from abc import ABC, abstractmethod

# Clase abstracta (no se puede instanciar directamente)
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side
```

ABSTRACCIÓN

Enfocarse en lo esencial de un objeto y ocultar los detalles innecesarios

- Tú solo sabes que puedes hacer una llamada, mandar un mensaje, sacar una foto.
- No sabes ni necesitas saber cómo funciona el chip, la antena o la memoria interna.

```
class Animal(ABC):  
    @abstractmethod  
    def make_sound(self):  
        pass
```

```
class Dog(Animal):  
    def make_sound(self):  
        return "Woof!"
```

```
class Cat(Animal):  
    def make_sound(self):  
        return "Meow!"
```



ENCAPSULACIÓN

30 SEP 2025

<https://github.com/juanvarco95>

ENCAPSULACIÓN

Ocultar los datos internos de un objeto y controlar cómo se accede a ellos

- Protege los atributos internos de modificaciones externas indebidas.
- Solo se puede interactuar con ellos a través de métodos públicos (getters, setters, funciones específicas).

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance # atributo privado

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        return self.__balance

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Not enough funds")
        return self.__balance

    def get_balance(self):
        return self.__balance
```

ENCAPSULACIÓN

Ocultar los datos internos de un objeto y controlar cómo se accede a ellos

- Encapsulamiento = protección de datos internos.
- Interacción solo mediante métodos públicos (getters y setters).
- Evita que los objetos queden en un estado inválido o inseguro.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.__price = price

    @property
    def price(self): # getter
        return self.__price

    @price.setter
    def price(self, new_price): # setter
        if new_price > 0:
            self.__price = new_price
        else:
            print("Price must be positive")
```

ENCAPSULACIÓN

Ocultar los datos internos de un objeto y controlar cómo se accede a ellos

- @property → permite leer el atributo como si fuera público, pero en realidad está protegido.
- @price.setter → define las reglas para modificar el atributo.
- @price.deleter → controla qué pasa si intentamos eliminar el atributo.

```
class Student:
    def __init__(self, name, grade):
        self.__name = name
        self.__grade = grade # private attribute

    # Getter
    @property
    def grade(self):
        return self.__grade

    # Setter with validation
    @grade.setter
    def grade(self, new_grade):
        if 0 <= new_grade <= 100:
            self.__grade = new_grade
        else:
            print("❌ Invalid grade. Must be between 0 and 100.")

    # Deleter
    @grade.deleter
    def grade(self):
        print(f"⚠️ Grade for {self.__name} cannot be deleted!")
```